DHBW Lörrach

Duale Hochschule Baden-Württemberg

Industrie 4.0 – Kanban – Unreal Engine

TIF23A – Jonas Götz, Marius Koch, Jonas Kroll, Elias Lanez, Nico Röcker – Gruppe D

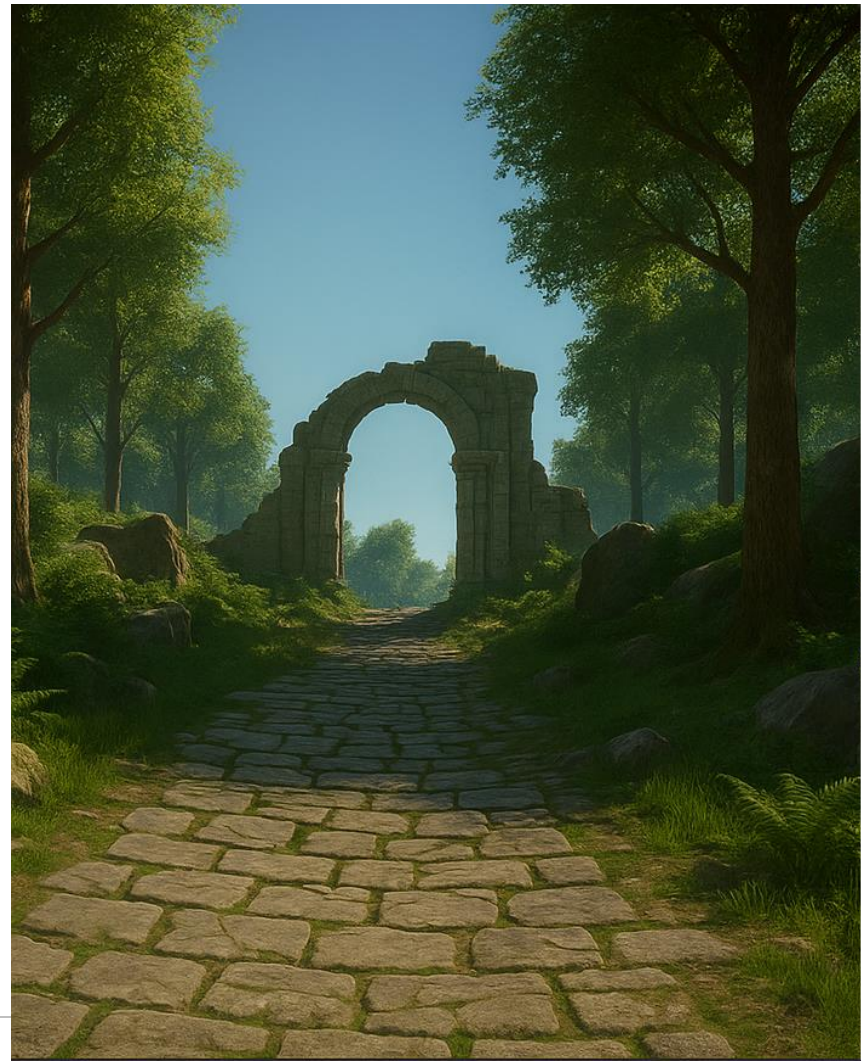
Projektmotivation

- Warum haben wir *Unreal Engine* gewählt?
- Warum das Thema *Industrie 4.0*?

Warum haben wir uns für Industrie 4.0 entschieden?

- Fokus auf Sensorik, Steuerung
- Realitätsnah an Praxis
- Interesse an Software- & Spieleentwicklung
- Digitaler Zwilling → “Digitales Einzelkind”
- Passend zum aktuellen Semester: Fokus auf IoT, Daten & Industrie 4.0

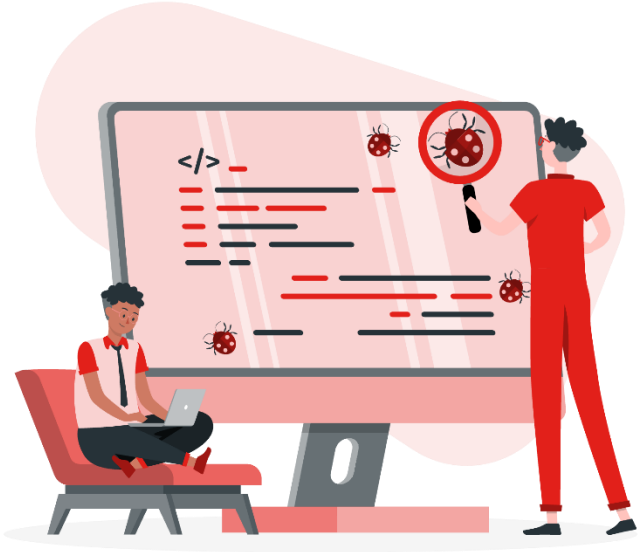
Warum haben wir Unreal Engine gewählt?



Quelle: Youtube (Stand 26.05.2025 11:00 Uhr)

Warum haben wir Unreal Engine gewählt?

- Realistische Darstellung
- Native VR-Unterstützung (z. B. Meta Quest 3)
- Blueprint-System: schnelle Prototypen ohne tiefes C++
- Hohe Performance & große Entwickler-Community
- Ideal für Simulation, Interaktion & visuelle Effekte
- Branchenstandard in moderner Spielentwicklung
- Interessant für uns durch Game-Entwicklung & Medienbezug



Herausforderung

- Start bei null – bewusst als Herausforderung gewählt
- Hoher Lerneffekt & persönliche Weiterentwicklung
- Unreal-Kenntnisse als langfristiger Vorteil für Beruf & private Projekte

Warum Kanban?

- Flexibler als Scrum
- Laufender Arbeitsfluss
- Weniger Meetings
- Selbstorganisation statt Hierarchie
- Volle Transparenz
- Besseres Aufgabenmanagment für kleinere Teams
- Eigenverantwortung

Projektmanagment

- Eigenverantwortliches Arbeiten mit klaren Rollen (z. B. Coach, SRM, SDM, TM)
- Hauptwerkzeug: Kanban-Board in Jira zur Aufgabenverteilung
- Fokus auf Flexibilität, Nachverfolgbarkeit und dynamische Priorisierung

Vorstellung des Teams

- Jonas Kroll: Entwickler
- Marius Koch: Service Request Manager
- Elias Lanez: Teammitglied; Service Delivery Manager
- Nico Röcker: Teammitglied
- Jonas Götz: Teammitglied; Coach

- M. Matt, E. Hanser: Stakeholder

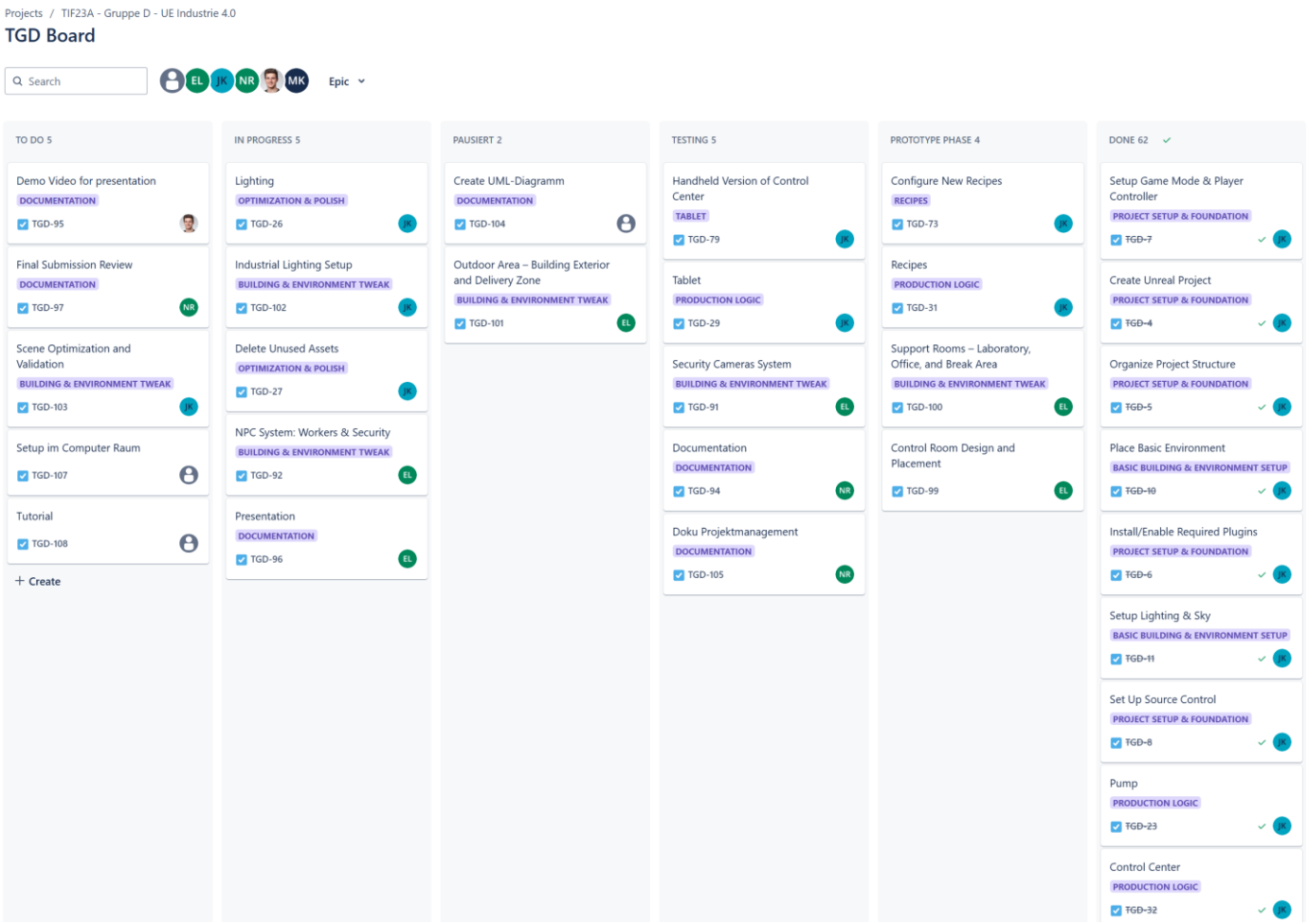
Aufbau und Organisation des Projekts

- Jira-Board mit 6 Spalten
- Aufgaben als Tickets mit Problem-/Lösungsstruktur
- Epics zur Gruppierung
- Kein Confluence genutzt
  - Klare Kommunikation über andere Kanäle
  - Zentralisierung in Jira ausreichend
  - Dokumentation direkt in Jira-Kommentaren oder externen Dateien

Unser Board

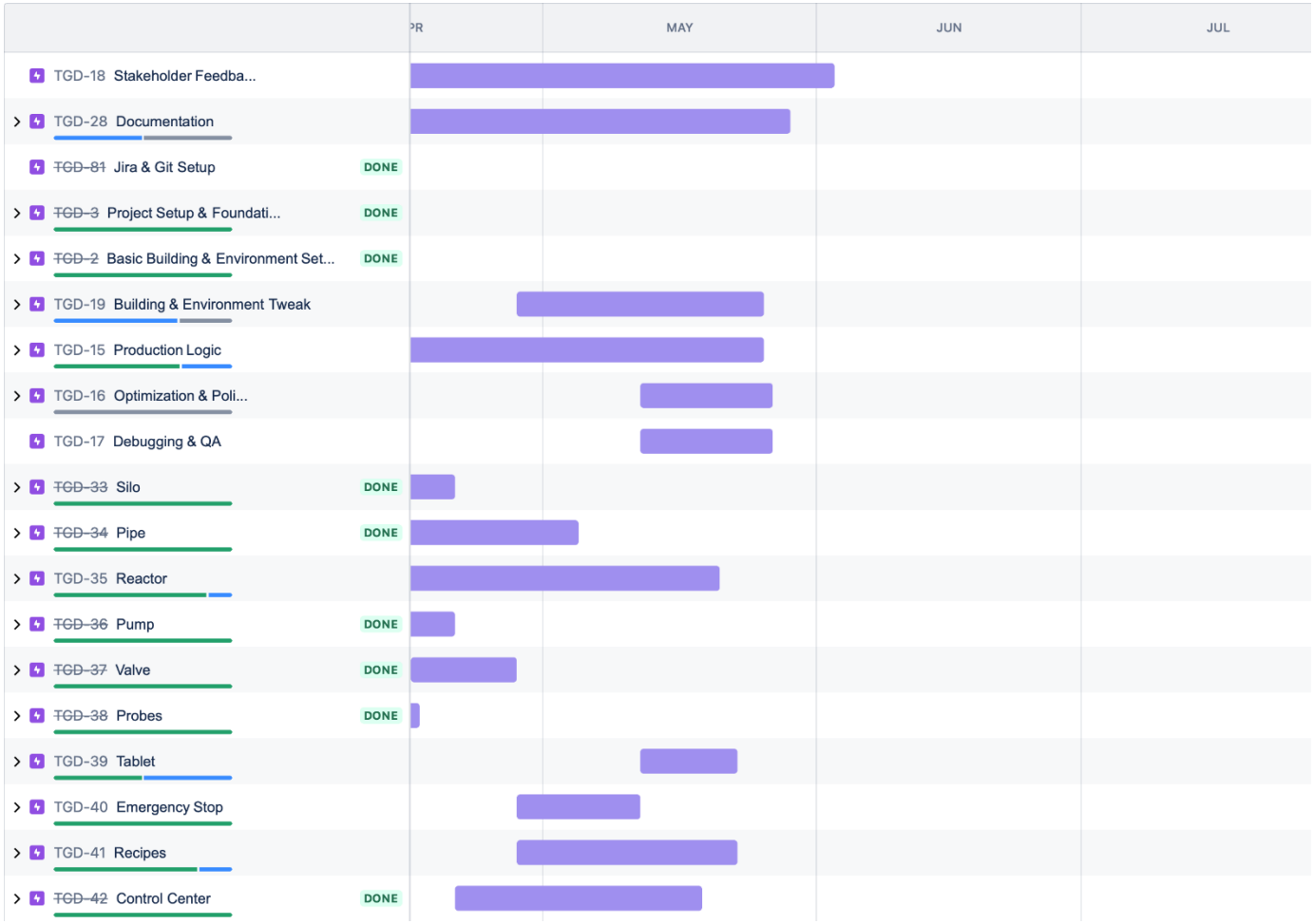
Warum hatten wir so viele Tickets (83)?

- **Transparenz**: Fortschritt und Status für alle jederzeit ersichtlich
- **Historie**: Nachvollziehbare Entwicklung mit Feedback-Dokumentation
- **Parallelisierung**: Gleichzeitiges Arbeiten an unabhängigen Modulen möglich
- **Qualität**: Jeder Task durchläuft Testing & Review



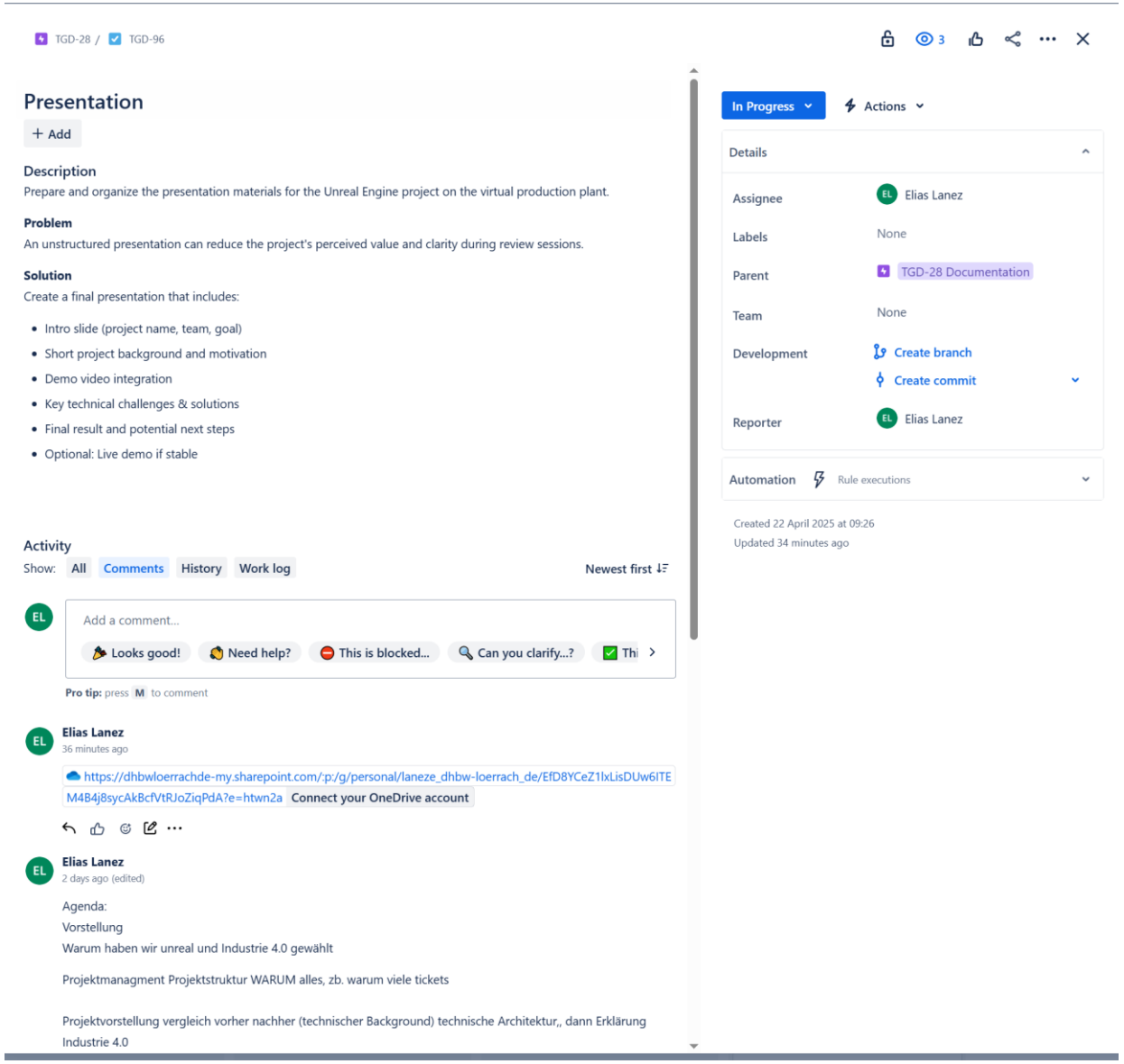
Epics

- Epics zur Gruppierung (z. B. Reaktor, UI, Doku)
- Klare Trennung von UI, Logik, Architektur & 3D-Design
- Bündelung größerer Aufgaben
- Planbarkeit (Zeitplan)
- Priorisierung

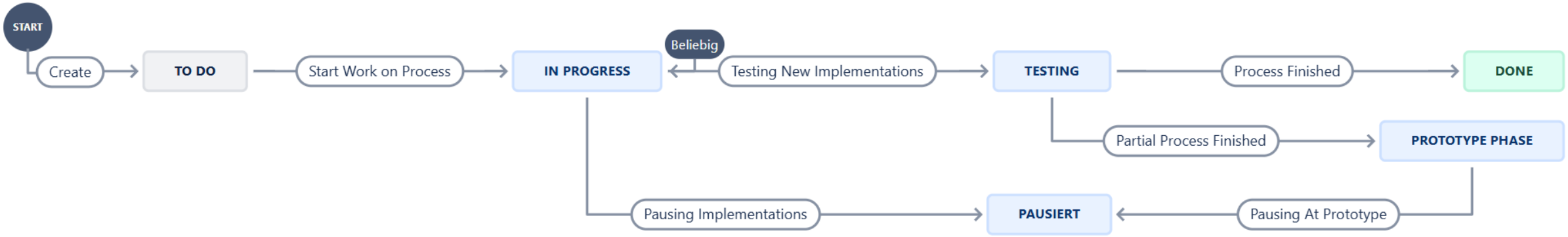


Ticket

- Aufgaben als Tickets mit Problem-Lösungsstruktur
- Klarheit bei Umsetzung



Workflow



- Klare Prozessstruktur
- Vermeidung halbfertiger Ergebnisse
- Separate Testphase sichert Qualität
- Transparenz für alle Beteiligten
- Bessere Nachvollziehbarkeit im Rückblick

Fazit zu Projektmanagment

Was lief gut?

- Klare Aufgabenstruktur
- Sinnvoller Implementierungsfluss
- Verlässliche Umsetzung
- Definierter Workflow
- Eigenverantwortliches Arbeiten

Was lief schlecht?

- Größe des Teams im Verhältnis zum Projekt
- Wenig Bedarf an zusätzlicher Abstimmung (Meetings)
- Gefühl von Overhead
- „Kanonen auf Spatzen“

Projektvorstellung: VORHER vs. Nachher

Projektvorstellung: Vorher vs. NACHHER

Vergleich reale Industrieanlage – unsere Industrieanlage

Highlights

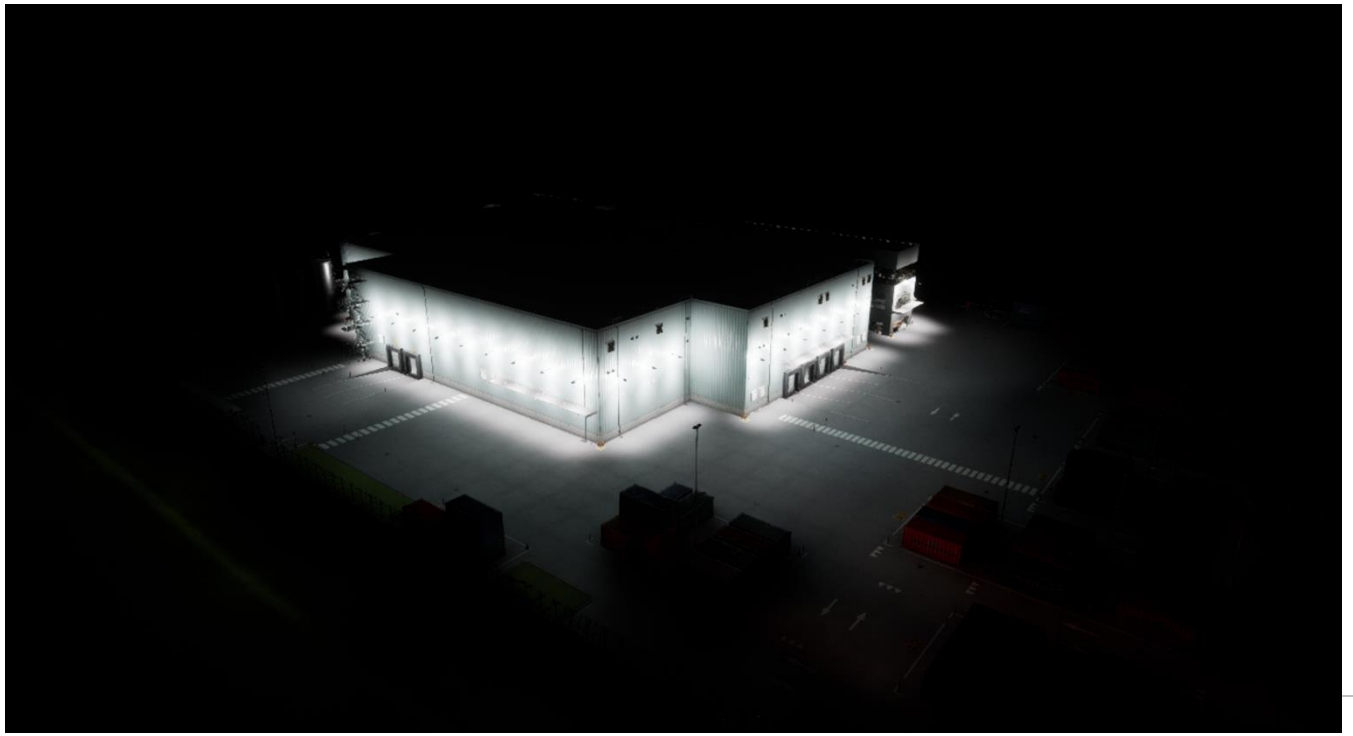
- Voll interaktive Anlage in VR:
- Steuerung über Monitor & Tablet (3D-UI)
- Rezeptverwaltung & automatisierte Abläufe
- CIP-Reinigung & Not-Aus-System
- Dynamischer Tag-Nacht-Zyklus zur Immersion
- Bürogebäude
- Forschungsanlage

Highlights

- Unreal Engine 5.4 mit OpenXR & Enhanced Input
- Modularisierung durch Blueprints: Komponenten wie BP_Reactor, BP_Silo, BP_Valve
- Zentraler FacilityDataManager (ABP_FacilityDataManager):
  - Koordiniert alle Komponenten
  - Automatisierte Steuerung & Logging
  - Implementiert Sicherheitslogik (Not-Aus)
- Observer-Pattern für UI-Updates in Echtzeit

Demo

- Video
- Falls Bedarf und Interesse besteht: Livedemo im Server-Raum



Fazit zu UE

- Kein C++ nur BluePrints keine neuen Programmierkenntnisse erforderlich
- Modularer Aufbau, Nutzerfreundlich
- Gute Skalierbarkeit
- Gute Assets, Bibliotheken und Tutorials
- Realisitische Graphik
- „Hassliebe“
- .exe - Livedemo